

GambleStyle

v1.0

Généré par Doxygen 1.9.1

1 GambleStyle	1
1.1 Structure du projet	1
1.2 Compilation	1
1.3 Classes principales	1
1.4 Tests	2
1.5 Documentation	2
2 Index hiérarchique	3
2.1 Hiérarchie des classes	3
3 Index des classes	5
3.1 Liste des classes	5
4 Index des fichiers	7
4.1 Liste des fichiers	7
5 Documentation des classes	9
5.1 Référence de la classe Carte	9
5.1.1 Description détaillée	9
5.1.2 Documentation des constructeurs et destructeur	9
5.1.2.1 Carte()	9
5.1.2.2 ~Carte()	9
5.1.3 Documentation des fonctions membres	10
5.1.3.1 action()	10
5.1.3.2 getId()	10
5.1.3.3 getNom()	10
5.1.3.4 getTexte()	10
5.1.4 Documentation des données membres	10
5.1.4.1 nom	10
5.1.4.2 texte	10
5.2 Référence de la classe CarteAmende	10
5.2.1 Description détaillée	11
5.2.2 Documentation des constructeurs et destructeur	11
5.2.2.1 CarteAmende()	11
5.2.3 Documentation des fonctions membres	11
5.2.3.1 action()	11
5.3 Référence de la classe CarteAnniversaire	11
5.3.1 Description détaillée	12
5.3.2 Documentation des constructeurs et destructeur	12
5.3.2.1 CarteAnniversaire()	12
5.3.3 Documentation des fonctions membres	12
5.3.3.1 action()	12
5.4 Référence de la classe CarteDeplacement	12
5.4.1 Description détaillée	13

5.4.2 Documentation des constructeurs et destructeur	13
5.4.2.1 CarteDeplacement()	13
5.4.3 Documentation des fonctions membres	13
5.4.3.1 action()	13
5.5 Référence de la classe CarteDoubleGain	13
5.5.1 Description détaillée	13
5.5.2 Documentation des constructeurs et destructeur	14
5.5.2.1 CarteDoubleGain()	14
5.5.3 Documentation des fonctions membres	14
5.5.3.1 action()	14
5.6 Référence de la classe CarteGain	14
5.6.1 Description détaillée	14
5.6.2 Documentation des constructeurs et destructeur	14
5.6.2.1 CarteGain()	14
5.6.3 Documentation des fonctions membres	15
5.6.3.1 action()	15
5.7 Référence de la classe CartelmpotMinijeu	15
5.7.1 Description détaillée	15
5.7.2 Documentation des constructeurs et destructeur	15
5.7.2.1 CartelmpotMinijeu()	15
5.7.3 Documentation des fonctions membres	15
5.7.3.1 action()	15
5.8 Référence de la classe CarteSortiePrison	16
5.8.1 Description détaillée	16
5.8.2 Documentation des constructeurs et destructeur	16
5.8.2.1 CarteSortiePrison()	16
5.8.3 Documentation des fonctions membres	16
5.8.3.1 action()	16
5.8.3.2 getId()	17
5.8.4 Documentation des données membres	17
5.8.4.1 ID	17
5.9 Référence de la classe Case	17
5.9.1 Description détaillée	17
5.9.2 Documentation des constructeurs et destructeur	17
5.9.2.1 Case() [1/2]	18
5.9.2.2 Case() [2/2]	18
5.9.2.3 ~Case()	18
5.9.3 Documentation des fonctions membres	18
5.9.3.1 action()	18
5.9.3.2 getIndex()	18
5.9.3.3 getJoueurActif()	18
5.9.3.4 getNom()	18

5.9.3.5 setIndex()	18
5.9.3.6 setJoueurActif()	19
5.9.4 Documentation des données membres	19
5.9.4.1 index	19
5.9.4.2 joueur_actif	19
5.9.4.3 nom	19
5.10 Référence de la classe CaseAleatoire	19
5.10.1 Description détaillée	20
5.10.2 Documentation des constructeurs et destructeur	20
5.10.2.1 CaseAleatoire()	20
5.10.2.2 ~CaseAleatoire()	20
5.10.3 Documentation des fonctions membres	20
5.10.3.1 action()	20
5.10.3.2 ajouterCarte()	20
5.10.3.3 getPremiereCarteDeFile()	20
5.11 Référence de la classe CaseDepart	20
5.11.1 Description détaillée	21
5.11.2 Documentation des constructeurs et destructeur	21
5.11.2.1 CaseDepart()	21
5.11.3 Documentation des fonctions membres	21
5.11.3.1 action()	21
5.11.3.2 getMontant()	21
5.12 Référence de la classe CaseEvenement	21
5.12.1 Description détaillée	22
5.12.2 Documentation des constructeurs et destructeur	22
5.12.2.1 CaseEvenement()	22
5.12.2.2 ~CaseEvenement()	22
5.12.3 Documentation des fonctions membres	22
5.12.3.1 action()	22
5.12.3.2 ajouterJoueur()	22
5.12.3.3 choixminijeu()	23
5.12.3.4 clearMinijeux()	23
5.12.3.5 distribution()	23
5.12.3.6 minijeuJoue()	23
5.12.3.7 setClassement()	23
5.12.3.8 setGagnant()	23
5.12.3.9 setMise()	23
5.13 Référence de la classe CasePrison	23
5.13.1 Description détaillée	24
5.13.2 Documentation des constructeurs et destructeur	24
5.13.2.1 CasePrison()	24
5.13.3 Documentation des fonctions membres	24

5.13.3.1 action()	24
5.13.3.2 emprisonner()	24
5.13.3.3 forcerResultatDes()	24
5.13.3.4 getTourPrison()	25
5.13.3.5 setUtiliserCarteReponse()	25
5.14 Référence de la classe CasePropriete	25
5.14.1 Description détaillée	25
5.14.2 Documentation des constructeurs et destructeur	26
5.14.2.1 CasePropriete()	26
5.14.3 Documentation des fonctions membres	26
5.14.3.1 achatPropose()	26
5.14.3.2 action()	26
5.14.3.3 gamble() [1 / 2]	26
5.14.3.4 gamble() [2 / 2]	26
5.14.3.5 getGamble()	26
5.14.3.6 getPrix()	26
5.14.3.7 getPrixHypothèque()	27
5.14.3.8 getProprietaire()	27
5.14.3.9 isAchetable()	27
5.14.3.10 minijeuLance()	27
5.14.3.11 repartition()	27
5.14.3.12 setAchetable()	27
5.14.3.13 setGagnant()	27
5.14.3.14 setGamble()	27
5.14.3.15 setGamblePredefini()	27
5.14.3.16 setMiseLimites()	28
5.14.3.17 setProprietaire()	28
5.15 Référence de la classe Joueur	28
5.15.1 Description détaillée	29
5.15.2 Documentation des constructeurs et destructeur	29
5.15.2.1 Joueur()	29
5.15.2.2 ~Joueur()	29
5.15.3 Documentation des fonctions membres	29
5.15.3.1 acheter()	29
5.15.3.2 bougerjoueur()	29
5.15.3.3 capitalSolvabilite()	29
5.15.3.4 clearProprietes()	30
5.15.3.5 conditionfinanciere()	30
5.15.3.6 donnerCapital()	30
5.15.3.7 donnerCarte()	30
5.15.3.8 enleverCapital()	30
5.15.3.9 getCapital()	30

5.15.3.10	getCaseActuelle()	30
5.15.3.11	getCompteurTours()	31
5.15.3.12	getDernierLancer()	31
5.15.3.13	getNbCartes()	31
5.15.3.14	getNbProprietes()	31
5.15.3.15	getNom()	31
5.15.3.16	getPosition()	31
5.15.3.17	getProprietes()	31
5.15.3.18	getValPropriete()	31
5.15.3.19	hypotequer()	32
5.15.3.20	lancerde()	32
5.15.3.21	misebanqueroute()	32
5.15.3.22	possedeCarte()	32
5.15.3.23	resetCompteurTours()	32
5.15.3.24	retirerCarte()	32
5.15.3.25	setCapital()	32
5.15.3.26	setDernierLancer()	32
5.15.3.27	setNbProprietesMock()	33
5.15.3.28	setPosition()	33
5.15.3.29	tour()	33
5.15.4	Documentation des données membres	33
5.15.4.1	capital	33
5.15.4.2	cartes	33
5.15.4.3	case_actuelle	33
5.15.4.4	compteur_double	33
5.15.4.5	compteur_tours	34
5.15.4.6	dernier_lancer	34
5.15.4.7	nb_proprietes_mock	34
5.15.4.8	nom	34
5.15.4.9	position	34
5.15.4.10	proprietes	34
5.15.4.11	use_mock_nb	34
5.15.4.12	val_propriete	34
5.16	Référence de la classe Minijeu	35
5.16.1	Description détaillée	35
5.16.2	Documentation des constructeurs et destructeur	35
5.16.2.1	Minijeu()	35
5.16.2.2	~Minijeu()	35
5.16.3	Documentation des fonctions membres	35
5.16.3.1	getNom()	35
5.16.3.2	jouer() [1/3]	36
5.16.3.3	jouer() [2/3]	36

5.16.3.4 jouer() [3/3]	36
5.17 Référence de la classe MinijeuTron	36
5.17.1 Description détaillée	37
5.17.2 Documentation des constructeurs et destructeur	37
5.17.2.1 MinijeuTron()	37
5.17.2.2 ~MinijeuTron()	37
5.17.3 Documentation des fonctions membres	37
5.17.3.1 getNom()	37
5.17.3.2 jouer()	37
5.18 Référence de la classe Plateau	38
5.18.1 Description détaillée	38
5.18.2 Documentation des constructeurs et destructeur	38
5.18.2.1 Plateau()	38
5.18.2.2 ~Plateau()	38
5.18.3 Documentation des fonctions membres	38
5.18.3.1 ajouterJoueur()	38
5.18.3.2 donnercapital()	38
5.18.3.3 enlevercapital()	39
5.18.3.4 fin()	39
5.18.3.5 finDePartie()	39
5.18.3.6 gamelooping()	39
5.18.3.7 getCase()	39
5.18.3.8 getJoueur()	39
5.18.3.9 getNbCases()	39
5.18.3.10 getNbJoueursActifs()	40
5.18.3.11 ordre()	40
5.18.3.12 retirerJoueursEnFaillite()	40
5.18.3.13 transfertargent()	40
5.18.3.14 trierParDernierLancer()	40
5.18.3.15 update()	40
6 Documentation des fichiers	41
6.1 Référence du fichier Carte.cpp	41
6.1.1 Description détaillée	41
6.2 Référence du fichier Carte.hpp	41
6.2.1 Description détaillée	41
6.3 Référence du fichier CarteAmende.cpp	41
6.3.1 Description détaillée	41
6.4 Référence du fichier CarteAmende.hpp	42
6.4.1 Description détaillée	42
6.5 Référence du fichier CarteAnniversaire.cpp	42
6.5.1 Description détaillée	42

6.6 Référence du fichier CarteAnniversaire.hpp	42
6.6.1 Description détaillée	42
6.7 Référence du fichier CarteDeplacement.cpp	42
6.7.1 Description détaillée	43
6.8 Référence du fichier CarteDeplacement.hpp	43
6.8.1 Description détaillée	43
6.9 Référence du fichier CarteDoubleGain.cpp	43
6.9.1 Description détaillée	43
6.10 Référence du fichier CarteDoubleGain.hpp	43
6.10.1 Description détaillée	43
6.11 Référence du fichier CarteGain.cpp	44
6.11.1 Description détaillée	44
6.12 Référence du fichier CarteGain.hpp	44
6.12.1 Description détaillée	44
6.13 Référence du fichier CartelImpotMinijeu.cpp	44
6.13.1 Description détaillée	44
6.14 Référence du fichier CartelImpotMinijeu.hpp	44
6.14.1 Description détaillée	45
6.15 Référence du fichier CarteSortiePrison.cpp	45
6.15.1 Description détaillée	45
6.16 Référence du fichier CarteSortiePrison.hpp	45
6.16.1 Description détaillée	45
6.17 Référence du fichier Case.cpp	45
6.17.1 Description détaillée	45
6.18 Référence du fichier Case.hpp	45
6.18.1 Description détaillée	46
6.19 Référence du fichier CaseAleatoire.cpp	46
6.19.1 Description détaillée	46
6.20 Référence du fichier CaseAleatoire.hpp	46
6.20.1 Description détaillée	46
6.21 Référence du fichier CaseDepart.cpp	46
6.21.1 Description détaillée	46
6.22 Référence du fichier CaseDepart.hpp	46
6.22.1 Description détaillée	47
6.23 Référence du fichier CaseEvenement.cpp	47
6.23.1 Description détaillée	47
6.24 Référence du fichier CaseEvenement.hpp	47
6.24.1 Description détaillée	47
6.25 Référence du fichier CasePrison.cpp	47
6.25.1 Description détaillée	48
6.26 Référence du fichier CasePrison.hpp	48
6.26.1 Description détaillée	48

6.27 Référence du fichier CasePropriete.cpp	48
6.27.1 Description détaillée	48
6.28 Référence du fichier CasePropriete.hpp	48
6.28.1 Description détaillée	48
6.29 Référence du fichier Condition.hpp	48
6.29.1 Description détaillée	49
6.29.2 Documentation du type de l'énumération	49
6.29.2.1 Condition	49
6.30 Référence du fichier Joueur.cpp	49
6.30.1 Description détaillée	49
6.31 Référence du fichier Joueur.hpp	49
6.31.1 Description détaillée	50
6.32 Référence du fichier main.cpp	50
6.32.1 Description détaillée	50
6.32.2 Documentation des fonctions	50
6.32.2.1 main()	50
6.33 Référence du fichier Minijeu.cpp	50
6.33.1 Description détaillée	50
6.34 Référence du fichier Minijeu.hpp	50
6.34.1 Description détaillée	51
6.35 Référence du fichier MinijeuTron.cpp	51
6.35.1 Description détaillée	51
6.36 Référence du fichier MinijeuTron.hpp	51
6.36.1 Description détaillée	51
6.37 Référence du fichier Plateau.cpp	51
6.37.1 Description détaillée	51
6.38 Référence du fichier Plateau.hpp	52
6.38.1 Description détaillée	52
6.39 Référence du fichier README.md	52

Index	53
--------------	-----------

Chapitre 1

GambleStyle

Jeu de plateau C++ pour 2 à 4 joueurs — Monopoly × Mario Party. **GM4** — INSA Rouen Normandie

LANDREAU Marius · DECHAMPS Gatien · HIS Adrien · GUILLOT Thomas

FEUILLOLAY Enora · FARIAS Samuel · LE Tuan Anh · PARAVEMAN Lucas

1.1 Structure du projet

```
GambleStyle/
|-- include/          <- Headers (.hpp) de toutes les classes
|-- src/              <- Implémentations (.cpp)
|-- ressource_codeur/ <- Tests unitaires CppUnit
|-- docs/             <- Documentation Doxygen (générée)
|-- assets/          <- Ressources graphiques SFML
|-- bin/              <- Exécutables (générés)
|-- obj/              <- Fichiers objets (générés)
|-- Doxyfile          <- Configuration Doxygen
|-- makefile          <- Compilation Make
|-- CMakeLists.txt    <- Compilation CMake
```

1.2 Compilation

```
# Avec Make
make          # Compile le jeu principal → bin/gamblestyle
make tests    # Compile et lance les tests CppUnit → bin/testEval
make docs     # Génère la documentation → docs/html/
make clean    # Supprime les .o
make remove   # Supprime tout (objets + exécutables)
# Avec CMake
mkdir build && cd build
cmake ..
make
```

Dépendances : g++ (C++11), CppUnit (tests), Doxygen (doc), SFML (mini-jeux graphiques)

1.3 Classes principales

Classe	Rôle
Joueur	Capital, propriétés, déplacements, condition financière
Plateau	Boucle de jeu, transferts d'argent, ordre des joueurs
Case	Abstraite — CaseDepart , CaseAleatoire , CasePropriete , CasePrison , CaseEvenement
MiniJeu	Interface mini-jeux (Tron, Pong, Puissance 4...)
Carte	Abstraite — 7 sous-types (Gain, Amende, Anniversaire...)

1.4 Tests

```
make tests  
# ou directement :  
./bin/testEval
```

Les résultats sont exportés dans `cppAllTestsResults.xml`.

1.5 Documentation

```
make docs  
# Ouvrir docs/html/index.html dans un navigateur
```

Chapitre 2

Index hiérarchique

2.1 Hiérarchie des classes

Cette liste d'héritage est classée approximativement par ordre alphabétique :

Carte	9
CarteAmende	10
CarteAnniversaire	11
CarteDeplacement	12
CarteDoubleGain	13
CarteGain	14
CarteImpotMinijeu	15
CarteSortiePrison	16
Case	17
CaseAleatoire	19
CaseDepart	20
CaseEvenement	21
CasePrison	23
CasePropriete	25
Joueur	28
Minijeu	35
MinijeuTron	36
Plateau	38

Chapitre 3

Index des classes

3.1 Liste des classes

Liste des classes, structures, unions et interfaces avec une brève description :

Carte	
Carte tirée sur une CaseAleatoire . Chaque sous-classe implémente action()	9
CarteAmende	
Retire un montant fixe du capital du joueur	10
CarteAnniversaire	
Chaque autre joueur verse un montant fixe au titulaire	11
CarteDeplacement	
Déplace un joueur vers une case spécifique	12
CarteDoubleGain	
Double les gains du joueur lors de sa prochaine victoire à un mini-jeu	13
CarteGain	
Crédite un montant fixe au capital du joueur	14
CarteImpotMinijeu	
Prélève $\text{tauxParCase} \times \text{nb_cases_possédées}$	15
CarteSortiePrison	
Permet au joueur de sortir de prison sans payer ni lancer les dés	16
Case	
Case abstraite du plateau. Chaque sous-classe implémente action()	17
CaseAleatoire	
Tire la première carte d'une file circulaire et l'applique	19
CaseDepart	
Crédite un montant fixe à chaque passage	20
CaseEvenement	
Déclenche un mini-jeu pour tous les joueurs. Distribution 3/4 – 1/4	21
CasePrison	
Bloque le joueur jusqu'à double, carte, ou 3 tours	23
CasePropriete	
Case achetable avec système de duel (gamble)	25
Joueur	
Représente un participant au jeu GambleStyle	28
Minijeu	
Mini-jeu jouable pour 2, 3 ou 4 joueurs	35
MinijeuTron	
Hérite de Minijeu . Ouvre une fenêtre SFML pour un duel 1v1	36
Plateau	
Chef d'orchestre de la partie GambleStyle	38

Chapitre 4

Index des fichiers

4.1 Liste des fichiers

Liste de tous les fichiers avec une brève description :

Carte.cpp	Implémentation de la classe de base Carte	41
Carte.hpp	Classe abstraite Carte	41
CarteAmende.cpp	Implémentation de CarteAmende	41
CarteAmende.hpp	Classe CarteAmende	42
CarteAnniversaire.cpp	Implémentation de CarteAnniversaire	42
CarteAnniversaire.hpp	Classe CarteAnniversaire	42
CarteDeplacement.cpp	Implémentation de CarteDeplacement	42
CarteDeplacement.hpp	Classe CarteDeplacement	43
CarteDoubleGain.cpp	Implémentation de CarteDoubleGain	43
CarteDoubleGain.hpp	Classe CarteDoubleGain	43
CarteGain.cpp	Implémentation de CarteGain	44
CarteGain.hpp	Classe CarteGain	44
CarteImpotMinijeu.cpp	Implémentation de CarteImpotMinijeu	44
CarteImpotMinijeu.hpp	Classe CarteImpotMinijeu	44
CarteSortiePrison.cpp	Implémentation de CarteSortiePrison	45
CarteSortiePrison.hpp	Classe CarteSortiePrison	45
Case.cpp	Implémentation de la classe de base Case	45
Case.hpp	Classe abstraite Case	45
CaseAleatoire.cpp	Implémentation de CaseAleatoire	46
CaseAleatoire.hpp	Classe CaseAleatoire	46

CaseDepart.cpp	
Implémentation de CaseDepart	46
CaseDepart.hpp	
Classe CaseDepart	46
CaseEvenement.cpp	
Implémentation de CaseEvenement	47
CaseEvenement.hpp	
Classe CaseEvenement	47
CasePrison.cpp	
Implémentation de CasePrison	47
CasePrison.hpp	
Classe CasePrison	48
CasePropriete.cpp	
Implémentation de CasePropriete	48
CasePropriete.hpp	
Classe CasePropriete	48
Condition.hpp	
Énumération de l'état financier d'un Joueur	48
Joueur.cpp	
Implémentation de la classe Joueur	49
Joueur.hpp	
Déclaration de la classe Joueur	49
main.cpp	
Point d'entrée du jeu GambleStyle	50
Minijeu.cpp	
Implémentation de Minijeu (version aléatoire)	50
Minijeu.hpp	
Classe Minijeu (implémentation par défaut aléatoire)	50
MinijeuTron.cpp	
Implémentation du mini-jeu Tron en SFML	51
MinijeuTron.hpp	
Classe MinijeuTron implémentant le jeu Tron via SFML	51
Plateau.cpp	
Implémentation de la classe Plateau	51
Plateau.hpp	
Déclaration de la classe Plateau	52

Chapitre 5

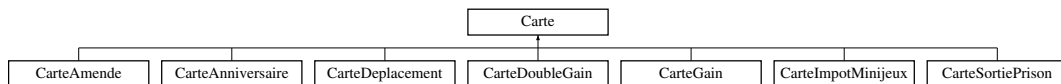
Documentation des classes

5.1 Référence de la classe Carte

[Carte](#) tirée sur une [CaseAleatoire](#). Chaque sous-classe implémente [action\(\)](#).

```
#include <Carte.hpp>
```

Graphe d'héritage de Carte:



Fonctions membres publiques

- [Carte](#) (const std::string &[nom](#), const std::string &[texte](#))
- virtual [~Carte](#) ()=default
- const std::string & [getNom](#) () const
- const std::string & [getTexte](#) () const
- virtual void [action](#) ()=0
- virtual int [getId](#) () const

Attributs protégés

- std::string [nom](#)
- std::string [texte](#)

5.1.1 Description détaillée

[Carte](#) tirée sur une [CaseAleatoire](#). Chaque sous-classe implémente [action\(\)](#).

Définition à la ligne 22 du fichier Carte.hpp.

5.1.2 Documentation des constructeurs et destructeur

5.1.2.1 Carte()

```
Carte::Carte (
    const std::string & nom,
    const std::string & texte )
```

Définition à la ligne 8 du fichier Carte.cpp.

5.1.2.2 ~Carte()

```
virtual Carte::~Carte ( ) [virtual], [default]
```

5.1.3 Documentation des fonctions membres

5.1.3.1 action()

```
virtual void Carte::action ( ) [pure virtual]
```

Implémenté dans [CarteSortiePrison](#), [CarteImpotMinijeu](#), [CarteGain](#), [CarteDoubleGain](#), [CarteDeplacement](#), [CarteAnniversaire](#), et [CarteAmende](#).

Référencé par `CaseAleatoire::action()`.

5.1.3.2 getId()

```
virtual int Carte::getId ( ) const [inline], [virtual]
```

Réimplémentée dans [CarteSortiePrison](#).

Définition à la ligne 31 du fichier `Carte.hpp`.

Référencé par `Joueur::donnerCarte()`.

5.1.3.3 getNom()

```
const std::string & Carte::getNom ( ) const
```

Définition à la ligne 11 du fichier `Carte.cpp`.

Références nom.

5.1.3.4 getTexte()

```
const std::string & Carte::getTexte ( ) const
```

Définition à la ligne 12 du fichier `Carte.cpp`.

Références texte.

5.1.4 Documentation des données membres

5.1.4.1 nom

```
std::string Carte::nom [protected]
```

Définition à la ligne 34 du fichier `Carte.hpp`.

Référencé par `getNom()`.

5.1.4.2 texte

```
std::string Carte::texte [protected]
```

Définition à la ligne 35 du fichier `Carte.hpp`.

Référencé par `getTexte()`.

La documentation de cette classe a été générée à partir du fichier suivant :

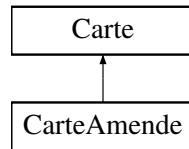
- [Carte.hpp](#)
- [Carte.cpp](#)

5.2 Référence de la classe CarteAmende

Retire un montant fixe du capital du joueur.

```
#include <CarteAmende.hpp>
```

Graphe d'héritage de `CarteAmende`:



Fonctions membres publiques

- [CarteAmende](#) ([Joueur](#) *joueur, [Plateau](#) *plateau, int montant)
- void [action](#) () override

Membres hérités additionnels

5.2.1 Description détaillée

Retire un montant fixe du capital du joueur.

Définition à la ligne 18 du fichier CarteAmende.hpp.

5.2.2 Documentation des constructeurs et destructeur

5.2.2.1 CarteAmende()

```

CarteAmende::CarteAmende (
    Joueur * joueur,
    Plateau * plateau,
    int montant )
  
```

Définition à la ligne 9 du fichier CarteAmende.cpp.

5.2.3 Documentation des fonctions membres

5.2.3.1 action()

```
void CarteAmende::action ( ) [override], [virtual]
```

Implémente [Carte](#).

Définition à la ligne 13 du fichier CarteAmende.cpp.

Références [Joueur::enleverCapital\(\)](#), et [Plateau::enlevercapital\(\)](#).

La documentation de cette classe a été générée à partir du fichier suivant :

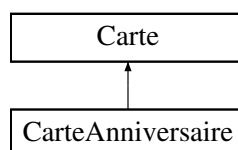
- [CarteAmende.hpp](#)
- [CarteAmende.cpp](#)

5.3 Référence de la classe CarteAnniversaire

Chaque autre joueur verse un montant fixe au titulaire.

```
#include <CarteAnniversaire.hpp>
```

Graphe d'héritage de CarteAnniversaire:



Fonctions membres publiques

- [CarteAnniversaire](#) ([Joueur](#) *titulaire, [Plateau](#) *plateau, int montant)
- void [action](#) () override

Membres hérités additionnels

5.3.1 Description détaillée

Chaque autre joueur verse un montant fixe au titulaire.
Définition à la ligne 18 du fichier CarteAnniversaire.hpp.

5.3.2 Documentation des constructeurs et destructeur

5.3.2.1 CarteAnniversaire()

```
CarteAnniversaire::CarteAnniversaire (
    Joueur * titulaire,
    Plateau * plateau,
    int montant )
```

Définition à la ligne 9 du fichier CarteAnniversaire.cpp.

5.3.3 Documentation des fonctions membres

5.3.3.1 action()

```
void CarteAnniversaire::action ( ) [override], [virtual]
```

Implémente [Carte](#).

Définition à la ligne 13 du fichier CarteAnniversaire.cpp.

Références [Plateau::getJoueur\(\)](#), [Plateau::getNbJoueursActifs\(\)](#), et [Plateau::transfertargent\(\)](#).

La documentation de cette classe a été générée à partir du fichier suivant :

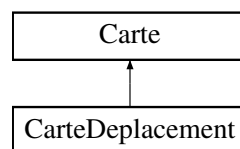
- [CarteAnniversaire.hpp](#)
- [CarteAnniversaire.cpp](#)

5.4 Référence de la classe CarteDeplacement

Déplace un joueur vers une case spécifique.

```
#include <CarteDeplacement.hpp>
```

Graphe d'héritage de CarteDeplacement:



Fonctions membres publiques

- [CarteDeplacement](#) ([Joueur](#) *joueur, [Plateau](#) *plateau, int caseIndex)
- void [action](#) () override

Membres hérités additionnels

5.4.1 Description détaillée

Déplace un joueur vers une case spécifique.

Définition à la ligne 18 du fichier CarteDeplacement.hpp.

5.4.2 Documentation des constructeurs et destructeur

5.4.2.1 CarteDeplacement()

```
CarteDeplacement::CarteDeplacement (
    Joueur * joueur,
    Plateau * plateau,
    int caseIndex )
```

Définition à la ligne 11 du fichier CarteDeplacement.cpp.

5.4.3 Documentation des fonctions membres

5.4.3.1 action()

```
void CarteDeplacement::action ( ) [override], [virtual]
```

Implémente [Carte](#).

Définition à la ligne 15 du fichier CarteDeplacement.cpp.

Références [Joueur::bougerjoueur\(\)](#), [Plateau::getCase\(\)](#), et [Joueur::setPosition\(\)](#).

La documentation de cette classe a été générée à partir du fichier suivant :

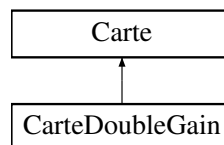
- [CarteDeplacement.hpp](#)
- [CarteDeplacement.cpp](#)

5.5 Référence de la classe CarteDoubleGain

Double les gains du joueur lors de sa prochaine victoire à un mini-jeu.

```
#include <CarteDoubleGain.hpp>
```

Graphe d'héritage de CarteDoubleGain:



Fonctions membres publiques

- [CarteDoubleGain](#) ([Joueur](#) *joueur)
- void [action](#) () override

Membres hérités additionnels

5.5.1 Description détaillée

Double les gains du joueur lors de sa prochaine victoire à un mini-jeu.

Définition à la ligne 18 du fichier CarteDoubleGain.hpp.

5.5.2 Documentation des constructeurs et destructeur

5.5.2.1 CarteDoubleGain()

```
CarteDoubleGain::CarteDoubleGain (
    Joueur * joueur )
```

Définition à la ligne 8 du fichier CarteDoubleGain.cpp.

5.5.3 Documentation des fonctions membres

5.5.3.1 action()

```
void CarteDoubleGain::action ( ) [override], [virtual]
```

Implémente [Carte](#).

Définition à la ligne 12 du fichier CarteDoubleGain.cpp.

Références [Joueur::donnerCarte\(\)](#).

La documentation de cette classe a été générée à partir du fichier suivant :

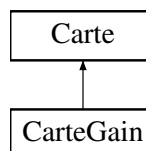
- [CarteDoubleGain.hpp](#)
- [CarteDoubleGain.cpp](#)

5.6 Référence de la classe CarteGain

Crédite un montant fixe au capital du joueur.

```
#include <CarteGain.hpp>
```

Graphe d'héritage de CarteGain:



Fonctions membres publiques

- [CarteGain](#) ([Joueur](#) *joueur, [Plateau](#) *plateau, int montant)
- void [action](#) () override

Membres hérités additionnels

5.6.1 Description détaillée

Crédite un montant fixe au capital du joueur.

Définition à la ligne 18 du fichier CarteGain.hpp.

5.6.2 Documentation des constructeurs et destructeur

5.6.2.1 CarteGain()

```
CarteGain::CarteGain (
    Joueur * joueur,
    Plateau * plateau,
    int montant )
```

Définition à la ligne 9 du fichier CarteGain.cpp.

5.6.3 Documentation des fonctions membres

5.6.3.1 action()

`void CarteGain::action ( ) [override], [virtual]`

Implémente [Carte](#).

Définition à la ligne 13 du fichier CarteGain.cpp.

Références `Joueur::donnerCapital()`, et `Plateau::donnercapital()`.

La documentation de cette classe a été générée à partir du fichier suivant :

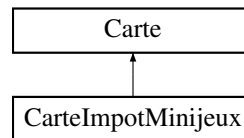
- [CarteGain.hpp](#)
- [CarteGain.cpp](#)

5.7 Référence de la classe CarteImpotMinijoux

Prélève `tauxParCase × nb_cases_possédées`.

`#include <CarteImpotMinijoux.hpp>`

Graphe d'héritage de `CarteImpotMinijoux`:



Fonctions membres publiques

- [CarteImpotMinijoux](#) ([Joueur](#) *joueur, [Plateau](#) *plateau, int tauxParCase=25)
- void [action](#) () override

Membres hérités additionnels

5.7.1 Description détaillée

Prélève `tauxParCase × nb_cases_possédées`.

Définition à la ligne 18 du fichier `CarteImpotMinijoux.hpp`.

5.7.2 Documentation des constructeurs et destructeur

5.7.2.1 CarteImpotMinijoux()

```

CarteImpotMinijoux::CarteImpotMinijoux (
    Joueur * joueur,
    Plateau * plateau,
    int tauxParCase = 25 )
  
```

Définition à la ligne 9 du fichier `CarteImpotMinijoux.cpp`.

5.7.3 Documentation des fonctions membres

5.7.3.1 action()

`void CarteImpotMinijoux::action ( ) [override], [virtual]`

Implémente [Carte](#).

Définition à la ligne 13 du fichier CarteImpotMinijeu.cpp.

Références Joueur::enleverCapital(), Plateau::enlevercapital(), et Joueur::getNbProprietes().

La documentation de cette classe a été générée à partir du fichier suivant :

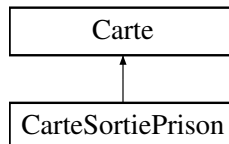
- [CarteImpotMinijeu.hpp](#)
- [CarteImpotMinijeu.cpp](#)

5.8 Référence de la classe CarteSortiePrison

Permet au joueur de sortir de prison sans payer ni lancer les dés.

```
#include <CarteSortiePrison.hpp>
```

Graphes d'héritage de CarteSortiePrison:



Fonctions membres publiques

- [CarteSortiePrison](#) ([Joueur](#) *joueur)
- void [action](#) () override
- int [getId](#) () const override

Attributs publics statiques

- static const int [ID](#) = 101

Membres hérités additionnels

5.8.1 Description détaillée

Permet au joueur de sortir de prison sans payer ni lancer les dés.

Définition à la ligne 18 du fichier CarteSortiePrison.hpp.

5.8.2 Documentation des constructeurs et destructeur

5.8.2.1 CarteSortiePrison()

```
CarteSortiePrison::CarteSortiePrison (
    Joueur * joueur )
```

Définition à la ligne 8 du fichier CarteSortiePrison.cpp.

5.8.3 Documentation des fonctions membres

5.8.3.1 action()

```
void CarteSortiePrison::action ( ) [override], [virtual]
```

Implémente [Carte](#).

Définition à la ligne 12 du fichier CarteSortiePrison.cpp.

Références Joueur::donnerCarte(), [ID](#), et Joueur::possedeCarte().

5.8.3.2 getId()

```
int CarteSortiePrison::getId ( ) const [inline], [override], [virtual]
```

Réimplémentée à partir de [Carte](#).

Définition à la ligne 23 du fichier CarteSortiePrison.hpp.

Références ID.

5.8.4 Documentation des données membres

5.8.4.1 ID

```
const int CarteSortiePrison::ID = 101 [static]
```

Définition à la ligne 20 du fichier CarteSortiePrison.hpp.

Référencé par action(), CasePrison::action(), et getId().

La documentation de cette classe a été générée à partir du fichier suivant :

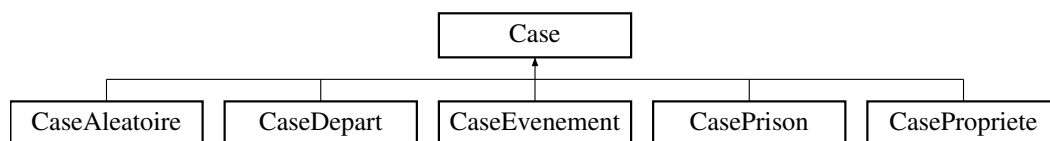
- [CarteSortiePrison.hpp](#)
- [CarteSortiePrison.cpp](#)

5.9 Référence de la classe Case

[Case](#) abstraite du plateau. Chaque sous-classe implémente [action\(\)](#).

```
#include <Case.hpp>
```

Graphe d'héritage de Case:



Fonctions membres publiques

- [Case](#) (const std::string &nom, int index)
- [Case](#) (int index)
- *Constructeur minimal (nom vide)*
- virtual [~Case](#) ()=default
- const std::string & [getNom](#) () const
- int [getIndex](#) () const
- void [setIndex](#) (int newIndex)
- void [setJoueurActif](#) ([Joueur](#) *j)
- [Joueur](#) * [getJoueurActif](#) () const
- virtual void [action](#) ()=0
- *Déclenche l'effet de la case sur le joueur actif.*

Attributs protégés

- std::string nom
- int index
- [Joueur](#) * joueur_actif

5.9.1 Description détaillée

[Case](#) abstraite du plateau. Chaque sous-classe implémente [action\(\)](#).

Définition à la ligne 20 du fichier Case.hpp.

5.9.2 Documentation des constructeurs et destructeur

5.9.2.1 Case() [1/2]

```
Case::Case (
    const std::string & nom,
    int index )
```

Définition à la ligne 7 du fichier Case.cpp.

5.9.2.2 Case() [2/2]

```
Case::Case (
    int index ) [explicit]
```

Constructeur minimal (nom vide)

Définition à la ligne 10 du fichier Case.cpp.

5.9.2.3 ~Case()

```
virtual Case::~Case ( ) [virtual], [default]
```

5.9.3 Documentation des fonctions membres

5.9.3.1 action()

```
virtual void Case::action ( ) [pure virtual]
```

Déclenche l'effet de la case sur le joueur actif.

Implémenté dans [CasePropriete](#), [CasePrison](#), [CaseEvenement](#), [CaseDepart](#), et [CaseAleatoire](#).

5.9.3.2 getIndex()

```
int Case::getIndex ( ) const
```

Définition à la ligne 14 du fichier Case.cpp.

Références index.

Référencé par Joueur::bougerjoueur().

5.9.3.3 getJoueurActif()

```
Joueur * Case::getJoueurActif ( ) const
```

Définition à la ligne 19 du fichier Case.cpp.

Références joueur_actif.

5.9.3.4 getNom()

```
const std::string & Case::getNom ( ) const
```

Définition à la ligne 13 du fichier Case.cpp.

Références nom.

5.9.3.5 setIndex()

```
void Case::setIndex (
    int newIndex )
```

Définition à la ligne 15 du fichier Case.cpp.

Références index.

5.9.3.6 setJoueurActif()

```
void Case::setJoueurActif (
    Joueur * j )
```

Définition à la ligne 18 du fichier Case.cpp.

Références joueur_actif.

Référencé par Joueur::bougerjoueur().

5.9.4 Documentation des données membres

5.9.4.1 index

```
int Case::index [protected]
```

Définition à la ligne 38 du fichier Case.hpp.

Référencé par getIndex(), et setIndex().

5.9.4.2 joueur_actif

```
Joueur* Case::joueur_actif [protected]
```

Définition à la ligne 39 du fichier Case.hpp.

Référencé par CaseDepart::action(), CasePrison::action(), CasePropriete::action(), getJoueurActif(), CasePropriete::repartition(), et setJoueurActif().

5.9.4.3 nom

```
std::string Case::nom [protected]
```

Définition à la ligne 37 du fichier Case.hpp.

Référencé par getNom().

La documentation de cette classe a été générée à partir du fichier suivant :

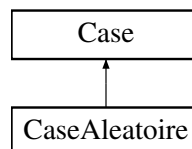
- [Case.hpp](#)
- [Case.cpp](#)

5.10 Référence de la classe CaseAleatoire

Tire la première carte d'une file circulaire et l'applique.

```
#include <CaseAleatoire.hpp>
```

Graphe d'héritage de CaseAleatoire:



Fonctions membres publiques

- [CaseAleatoire \(\)](#)
 - virtual [~CaseAleatoire \(\)](#)
 - void [ajouterCarte \(Carte *c\)](#)
 - [Carte * getPremiereCarteDeFile \(\)](#) const
 - void [action \(\)](#) override
- Déclenche l'effet de la case sur le joueur actif.*

Membres hérités additionnels

5.10.1 Description détaillée

Tire la première carte d'une file circulaire et l'applique.
Définition à la ligne 21 du fichier CaseAleatoire.hpp.

5.10.2 Documentation des constructeurs et destructeur

5.10.2.1 CaseAleatoire()

```
CaseAleatoire::CaseAleatoire ( )
```

Définition à la ligne 8 du fichier CaseAleatoire.cpp.

5.10.2.2 ~CaseAleatoire()

```
CaseAleatoire::~~CaseAleatoire ( ) [virtual]
```

Définition à la ligne 10 du fichier CaseAleatoire.cpp.

5.10.3 Documentation des fonctions membres

5.10.3.1 action()

```
void CaseAleatoire::action ( ) [override], [virtual]
```

Déclenche l'effet de la case sur le joueur actif.
Implémente [Case](#).
Définition à la ligne 25 du fichier CaseAleatoire.cpp.
Références [Carte::action\(\)](#).

5.10.3.2 ajouterCarte()

```
void CaseAleatoire::ajouterCarte (
    Carte * c )
```

Définition à la ligne 17 du fichier CaseAleatoire.cpp.

5.10.3.3 getPremiereCarteDeFile()

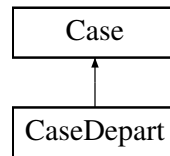
```
Carte * CaseAleatoire::getPremiereCarteDeFile ( ) const
```

Définition à la ligne 21 du fichier CaseAleatoire.cpp.
La documentation de cette classe a été générée à partir du fichier suivant :

- [CaseAleatoire.hpp](#)
- [CaseAleatoire.cpp](#)

5.11 Référence de la classe CaseDepart

Crédite un montant fixe à chaque passage.
`#include <CaseDepart.hpp>`
Graphe d'héritage de CaseDepart:



Fonctions membres publiques

- [CaseDepart](#) (int montant)
- void [action](#) () override
Déclenche l'effet de la case sur le joueur actif.
- int [getMontant](#) () const

Membres hérités additionnels

5.11.1 Description détaillée

Crédite un montant fixe à chaque passage.
Définition à la ligne 18 du fichier CaseDepart.hpp.

5.11.2 Documentation des constructeurs et destructeur

5.11.2.1 CaseDepart()

```
CaseDepart::CaseDepart (
    int montant ) [explicit]
```

Définition à la ligne 8 du fichier CaseDepart.cpp.

5.11.3 Documentation des fonctions membres

5.11.3.1 action()

```
void CaseDepart::action ( ) [override], [virtual]
```

Déclenche l'effet de la case sur le joueur actif.
Implémente [Case](#).
Définition à la ligne 11 du fichier CaseDepart.cpp.
Références [Joueur::donnerCapital\(\)](#), et [Case::joueur_actif](#).

5.11.3.2 getMontant()

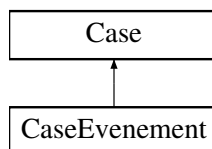
```
int CaseDepart::getMontant ( ) const
```

Définition à la ligne 15 du fichier CaseDepart.cpp.
La documentation de cette classe a été générée à partir du fichier suivant :

- [CaseDepart.hpp](#)
- [CaseDepart.cpp](#)

5.12 Référence de la classe CaseEvenement

Déclenche un mini-jeu pour tous les joueurs. Distribution 3/4 – 1/4.
`#include <CaseEvenement.hpp>`
Graphe d'héritage de CaseEvenement:



Fonctions membres publiques

- `CaseEvenement()`
- `virtual ~CaseEvenement()`
- `void action()` override
Déclenche l'effet de la case sur le joueur actif.
- `void ajouterJoueur(Joueur *j)`
- `void setMise(int m)`
- `void setGagnant(Joueur *j)`
- `void setClassement(const std::vector< Joueur * > &cl)`
- `void clearMinijeu()`
- `Minijeu * choixminijeu()`
- `void distribution()`
- `bool minijeuJoue()` const

Membres hérités additionnels

5.12.1 Description détaillée

Déclenche un mini-jeu pour tous les joueurs. Distribution 3/4 – 1/4.
 Définition à la ligne 21 du fichier CaseEvenement.hpp.

5.12.2 Documentation des constructeurs et destructeur

5.12.2.1 CaseEvenement()

```
CaseEvenement::CaseEvenement ( )
```

Définition à la ligne 11 du fichier CaseEvenement.cpp.

5.12.2.2 ~CaseEvenement()

```
CaseEvenement::~~CaseEvenement ( ) [virtual]
```

Définition à la ligne 18 du fichier CaseEvenement.cpp.

5.12.3 Documentation des fonctions membres

5.12.3.1 action()

```
void CaseEvenement::action ( ) [override], [virtual]
```

Déclenche l'effet de la case sur le joueur actif.

Implémente [Case](#).

Définition à la ligne 61 du fichier CaseEvenement.cpp.

Références `choixminijeu()`, `distribution()`, et `Minijeu::jouer()`.

5.12.3.2 ajouterJoueur()

```
void CaseEvenement::ajouterJoueur (
```

```
    Joueur * j )
```

Définition à la ligne 26 du fichier CaseEvenement.cpp.

5.12.3.3 choixminijeu()

`Minijeu * CaseEvenement::choixminijeu ( )`

Définition à la ligne 41 du fichier CaseEvenement.cpp.

Référencé par action().

5.12.3.4 clearMinijeux()

`void CaseEvenement::clearMinijeux ( )`

Définition à la ligne 29 du fichier CaseEvenement.cpp.

5.12.3.5 distribution()

`void CaseEvenement::distribution ( )`

Définition à la ligne 47 du fichier CaseEvenement.cpp.

Référencé par action().

5.12.3.6 minijeuJoue()

`bool CaseEvenement::minijeuJoue ( ) const`

Définition à la ligne 35 du fichier CaseEvenement.cpp.

5.12.3.7 setClassement()

`void CaseEvenement::setClassement (`
`const std::vector< Joueur * > & cl )`

Définition à la ligne 37 du fichier CaseEvenement.cpp.

5.12.3.8 setGagnant()

`void CaseEvenement::setGagnant (`
`Joueur * j )`

Définition à la ligne 28 du fichier CaseEvenement.cpp.

5.12.3.9 setMise()

`void CaseEvenement::setMise (`
`int m )`

Définition à la ligne 27 du fichier CaseEvenement.cpp.

La documentation de cette classe a été générée à partir du fichier suivant :

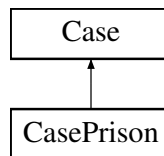
- [CaseEvenement.hpp](#)
- [CaseEvenement.cpp](#)

5.13 Référence de la classe CasePrison

Bloque le joueur jusqu'à double, carte, ou 3 tours.

```
#include <CasePrison.hpp>
```

Graphe d'héritage de CasePrison:



Fonctions membres publiques

- [CasePrison](#) ()
- void [action](#) () override
Déclenche l'effet de la case sur le joueur actif.
- void [emprisonner](#) ([Joueur](#) *)
- int [getTourPrison](#) ([Joueur](#) *) const
- void [forcerResultatDes](#) (int d1, int d2)
Pour les tests : force le résultat des dés.
- void [setUtiliserCarteReponse](#) (bool rep)
Pour les tests : simule la réponse du joueur pour la carte.

Membres hérités additionnels

5.13.1 Description détaillée

Bloque le joueur jusqu'à double, carte, ou 3 tours.
Définition à la ligne 20 du fichier CasePrison.hpp.

5.13.2 Documentation des constructeurs et destructeur

5.13.2.1 CasePrison()

`CasePrison::CasePrison ( )`
Définition à la ligne 10 du fichier CasePrison.cpp.

5.13.3 Documentation des fonctions membres

5.13.3.1 action()

`void CasePrison::action ( ) [override], [virtual]`
Déclenche l'effet de la case sur le joueur actif.
Implémente [Case](#).
Définition à la ligne 29 du fichier CasePrison.cpp.
Références `CarteSortiePrison::ID`, `Case::joueur_actif`, `Joueur::possedeCarte()`, et `Joueur::retirerCarte()`.

5.13.3.2 emprisonner()

`void CasePrison::emprisonner (`
 [Joueur](#) * j)
Définition à la ligne 13 du fichier CasePrison.cpp.

5.13.3.3 forcerResultatDes()

`void CasePrison::forcerResultatDes (`
 int d1,
 int d2)
Pour les tests : force le résultat des dés.

Définition à la ligne 26 du fichier CasePrison.cpp.

5.13.3.4 getTourPrison()

```
int CasePrison::getTourPrison (
    Joueur * j ) const
```

Définition à la ligne 21 du fichier CasePrison.cpp.

5.13.3.5 setUtiliserCarteReponse()

```
void CasePrison::setUtiliserCarteReponse (
    bool rep )
```

Pour les tests : simule la réponse du joueur pour la carte.

Définition à la ligne 27 du fichier CasePrison.cpp.

La documentation de cette classe a été générée à partir du fichier suivant :

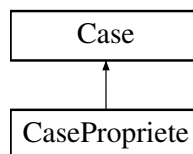
- [CasePrison.hpp](#)
- [CasePrison.cpp](#)

5.14 Référence de la classe CasePropriete

[Case](#) achetable avec système de duel (gamble).

```
#include <CasePropriete.hpp>
```

Graphe d'héritage de CasePropriete:



Fonctions membres publiques

- [CasePropriete](#) (const std::string &nom, int prix)
- void [action](#) () override
- *Déclenche l'effet de la case sur le joueur actif.*
- Joueur * [getProprietaire](#) () const
- int [getPrix](#) () const
- int [getPrixHypothèque](#) () const
- bool [isAchetable](#) () const
- bool [achatPropose](#) () const
- bool [minijeuLance](#) () const
- int [getGamble](#) () const
- void [setProprietaire](#) (Joueur *j)
- void [setAchetable](#) (bool v)
- void [setGamble](#) (int montant)
- void [setGagnant](#) (Joueur *j)
- void [setGamblePredefini](#) (int v)
- void [setMiseLimites](#) (int min, int max)
- void [gamble](#) ()
- void [gamble](#) (Joueur *visiteur)
- void [repartition](#) ()

Membres hérités additionnels

5.14.1 Description détaillée

[Case](#) achetable avec système de duel (gamble).

Définition à la ligne 19 du fichier CasePropriete.hpp.

5.14.2 Documentation des constructeurs et destructeur

5.14.2.1 CasePropriete()

```
CasePropriete::CasePropriete (
    const std::string & nom,
    int prix )
```

Définition à la ligne 8 du fichier CasePropriete.cpp.

5.14.3 Documentation des fonctions membres

5.14.3.1 achatPropose()

```
bool CasePropriete::achatPropose ( ) const
```

Définition à la ligne 44 du fichier CasePropriete.cpp.

5.14.3.2 action()

```
void CasePropriete::action ( ) [override], [virtual]
```

Déclenche l'effet de la case sur le joueur actif.

Implémente [Case](#).

Définition à la ligne 14 du fichier CasePropriete.cpp.

Références gamble(), et Case::joueur_actif.

5.14.3.3 gamble() [1/2]

```
void CasePropriete::gamble ( )
```

Définition à la ligne 24 du fichier CasePropriete.cpp.

Référencé par action(), et gamble().

5.14.3.4 gamble() [2/2]

```
void CasePropriete::gamble (
    Joueur * visiteur )
```

Définition à la ligne 31 du fichier CasePropriete.cpp.

Références gamble().

5.14.3.5 getGamble()

```
int CasePropriete::getGamble ( ) const
```

Définition à la ligne 46 du fichier CasePropriete.cpp.

5.14.3.6 getPrix()

```
int CasePropriete::getPrix ( ) const
```

Définition à la ligne 41 du fichier CasePropriete.cpp.

Référencé par Joueur::acheter().

5.14.3.7 getPrixHypothèque()

```
int CasePropriete::getPrixHypothèque ( ) const
```

Définition à la ligne 42 du fichier CasePropriete.cpp.
Référéncé par Joueur::acheter(), et Joueur::hypotequer().

5.14.3.8 getProprietaire()

```
Joueur * CasePropriete::getProprietaire ( ) const
```

Définition à la ligne 40 du fichier CasePropriete.cpp.
Référéncé par Joueur::acheter().

5.14.3.9 isAchetable()

```
bool CasePropriete::isAchetable ( ) const
```

Définition à la ligne 43 du fichier CasePropriete.cpp.

5.14.3.10 minijeuLance()

```
bool CasePropriete::minijeuLance ( ) const
```

Définition à la ligne 45 du fichier CasePropriete.cpp.

5.14.3.11 repartition()

```
void CasePropriete::repartition ( )
```

Définition à la ligne 33 du fichier CasePropriete.cpp.
Référénces Joueur::donnerCapital(), Joueur::enleverCapital(), et Case::joueur_actif.

5.14.3.12 setAchetable()

```
void CasePropriete::setAchetable (
    bool v )
```

Définition à la ligne 48 du fichier CasePropriete.cpp.
Référéncé par Joueur::acheter(), et Joueur::hypotequer().

5.14.3.13 setGagnant()

```
void CasePropriete::setGagnant (
    Joueur * j )
```

Définition à la ligne 50 du fichier CasePropriete.cpp.

5.14.3.14 setGamble()

```
void CasePropriete::setGamble (
    int montant )
```

Définition à la ligne 49 du fichier CasePropriete.cpp.

5.14.3.15 setGamblePredefini()

```
void CasePropriete::setGamblePredefini (
    int v )
```

Définition à la ligne 51 du fichier CasePropriete.cpp.

5.14.3.16 setMiseLimites()

```
void CasePropriete::setMiseLimites (
    int min,
    int max )
```

Définition à la ligne 52 du fichier CasePropriete.cpp.

5.14.3.17 setProprietaire()

```
void CasePropriete::setProprietaire (
    Joueur * j )
```

Définition à la ligne 47 du fichier CasePropriete.cpp.

Référencé par Joueur::acheter().

La documentation de cette classe a été générée à partir du fichier suivant :

- [CasePropriete.hpp](#)
- [CasePropriete.cpp](#)

5.15 Référence de la classe Joueur

Représente un participant au jeu GambleStyle.

```
#include <Joueur.hpp>
```

Fonctions membres publiques

- [Joueur](#) (const std::string &nom, int capital)
- virtual [~Joueur](#) ()=default
- const std::string & [getNom](#) () const
- int [getCapital](#) () const
- int [getValPropriete](#) () const
- int [getNbProprietes](#) () const
- [Case](#) * [getCaseActuelle](#) () const
- int [getDernierLancer](#) () const
- int [getCompteurTours](#) () const
- int [getNbCartes](#) () const
- int [getPosition](#) () const
- const std::vector< [CasePropriete](#) * > & [getProprietes](#) () const
- void [setCapital](#) (int c)
- void [setDernierLancer](#) (int v)
- void [resetCompteurTours](#) ()
- void [donnerCapital](#) (int montant)
- void [enleverCapital](#) (int montant)
- [Condition](#) [conditionfinanciere](#) () const
- int [capitalSolvabilite](#) () const
- void [misebanqueroute](#) ()
- void [acheter](#) ([CasePropriete](#) *caseProp)
- void [hypotequer](#) ([CasePropriete](#) *caseProp)
- void [clearProprietes](#) ()
- void [setNbProprietesMock](#) (int n)
- *Utilisé par [CarteImpotMinijeu](#) dans les tests.*
- void [donnerCarte](#) ([Carte](#) *carte)
- bool [possedeCarte](#) (int id) const
- void [retirerCarte](#) (int id)
- int [lancerde](#) ()
- virtual void [tour](#) ()
- void [bougerjoueur](#) ([Case](#) *dest)
- void [setPosition](#) (int index)

Attributs protégés

- std::string nom
- int capital
- int val_propriete
- int position
- int dernier_lancer

```

— int compteur_double
— int compteur_tours
— int nb_proprietes_mock
    Pour CartelmpotMinijeu tests.
— bool use_mock_nb
— std::vector< Carte * > cartes
— std::vector< CasePropriete * > proprietes
— Case * case_actuelle

```

5.15.1 Description détaillée

Représente un participant au jeu GambleStyle.
Définition à la ligne 21 du fichier Joueur.hpp.

5.15.2 Documentation des constructeurs et destructeur

5.15.2.1 Joueur()

```

Joueur::Joueur (
    const std::string & nom,
    int capital )

```

Définition à la ligne 12 du fichier Joueur.cpp.

5.15.2.2 ~Joueur()

```

virtual Joueur::~Joueur ( ) [virtual], [default]

```

5.15.3 Documentation des fonctions membres

5.15.3.1 acheter()

```

void Joueur::acheter (
    CasePropriete * caseProp )

```

Définition à la ligne 55 du fichier Joueur.cpp.

Références capitalSolvabilite(), enleverCapital(), CasePropriete::getPrix(), CasePropriete::getPrixHypothèque(), CasePropriete::getProprietaire(), proprietes, CasePropriete::setAchetable(), CasePropriete::setProprietaire(), et val_propriete.

5.15.3.2 bougerjoueur()

```

void Joueur::bougerjoueur (
    Case * dest )

```

Définition à la ligne 111 du fichier Joueur.cpp.

Références case_actuelle, Case::getIndex(), position, et Case::setJoueurActif().

Référencé par CarteDeplacement::action().

5.15.3.3 capitalSolvabilite()

```

int Joueur::capitalSolvabilite ( ) const

```

Définition à la ligne 48 du fichier Joueur.cpp.

Références capital, et val_propriete.

Référencé par acheter(), et Plateau::fin().

5.15.3.4 clearProprietes()

```
void Joueur::clearProprietes ( )
```

Définition à la ligne 96 du fichier Joueur.cpp.

Références proprietes, use_mock_nb, et val_propriete.

Référencé par misebanqueroute().

5.15.3.5 conditionfinanciere()

```
Condition Joueur::conditionfinanciere ( ) const
```

Définition à la ligne 42 du fichier Joueur.cpp.

Références BANQUEROUTE, capital, FAILLITE, RICHE, et val_propriete.

5.15.3.6 donnerCapital()

```
void Joueur::donnerCapital (
    int montant )
```

Définition à la ligne 39 du fichier Joueur.cpp.

Références capital.

Référencé par CarteGain::action(), CaseDepart::action(), Plateau::donnercapital(), hypotequer(), CasePropriete::repartition(), et Plateau::transfertargent().

5.15.3.7 donnerCarte()

```
void Joueur::donnerCarte (
    Carte * carte )
```

Définition à la ligne 103 du fichier Joueur.cpp.

Références cartes, Carte::getId(), et possedeCarte().

Référencé par CarteDoubleGain::action(), et CarteSortiePrison::action().

5.15.3.8 enleverCapital()

```
void Joueur::enleverCapital (
    int montant )
```

Définition à la ligne 40 du fichier Joueur.cpp.

Références capital.

Référencé par acheter(), CarteAmende::action(), CartelImpotMinijeu::action(), Plateau::enlevercapital(), CasePropriete::repartition(), et Plateau::transfertargent().

5.15.3.9 getCapital()

```
int Joueur::getCapital ( ) const
```

Définition à la ligne 20 du fichier Joueur.cpp.

Références capital.

5.15.3.10 getCaseActuelle()

```
Case * Joueur::getCaseActuelle ( ) const
```

Définition à la ligne 23 du fichier Joueur.cpp.

Références case_actuelle.

5.15.3.11 getCompteurTours()

```
int Joueur::getCompteurTours ( ) const
```

Définition à la ligne 25 du fichier Joueur.cpp.

Références compteur_tours.

5.15.3.12 getDernierLancer()

```
int Joueur::getDernierLancer ( ) const
```

Définition à la ligne 24 du fichier Joueur.cpp.

Références dernier_lancer.

5.15.3.13 getNbCartes()

```
int Joueur::getNbCartes ( ) const
```

Définition à la ligne 26 du fichier Joueur.cpp.

Références cartes.

5.15.3.14 getNbProprietes()

```
int Joueur::getNbProprietes ( ) const
```

Définition à la ligne 22 du fichier Joueur.cpp.

Références nb_proprietes_mock, proprietes, et use_mock_nb.

Référencé par CartelImpotMinijeu::action().

5.15.3.15 getNom()

```
const std::string & Joueur::getNom ( ) const
```

Définition à la ligne 19 du fichier Joueur.cpp.

Références nom.

Référencé par MinijeuTron::jouer(), et main().

5.15.3.16 getPosition()

```
int Joueur::getPosition ( ) const
```

Définition à la ligne 27 du fichier Joueur.cpp.

Références position.

5.15.3.17 getProprietes()

```
const std::vector< CasePropriete * > & Joueur::getProprietes ( ) const
```

Définition à la ligne 32 du fichier Joueur.cpp.

Références proprietes.

5.15.3.18 getValPropriete()

```
int Joueur::getValPropriete ( ) const
```

Définition à la ligne 21 du fichier Joueur.cpp.

Références val_propriete.

5.15.3.19 hypotequer()

```
void Joueur::hypotequer (
    CasePropriete * caseProp )
```

Définition à la ligne 85 du fichier Joueur.cpp.

Références donnerCapital(), CasePropriete::getPrixHypothèque(), proprietes, CasePropriete::setAchetable(), et val_propriete.

5.15.3.20 lancerde()

```
int Joueur::lancerde ( )
```

Définition à la ligne 109 du fichier Joueur.cpp.

5.15.3.21 misebanqueroute()

```
void Joueur::misebanqueroute ( )
```

Définition à la ligne 50 du fichier Joueur.cpp.

Références capital, et clearProprietes().

5.15.3.22 possedeCarte()

```
bool Joueur::possedeCarte (
    int id ) const
```

Définition à la ligne 104 du fichier Joueur.cpp.

Références cartes.

Référencé par CarteSortiePrison::action(), CasePrison::action(), et donnerCarte().

5.15.3.23 resetCompteurTours()

```
void Joueur::resetCompteurTours ( )
```

Définition à la ligne 37 du fichier Joueur.cpp.

Références compteur_tours.

5.15.3.24 retirerCarte()

```
void Joueur::retirerCarte (
    int id )
```

Définition à la ligne 105 du fichier Joueur.cpp.

Références cartes.

Référencé par CasePrison::action().

5.15.3.25 setCapital()

```
void Joueur::setCapital (
    int c )
```

Définition à la ligne 35 du fichier Joueur.cpp.

Références capital.

5.15.3.26 setDernierLancer()

```
void Joueur::setDernierLancer (
    int v )
```

Définition à la ligne 36 du fichier Joueur.cpp.

Références dernier_lancer.

5.15.3.27 setNbProprietesMock()

```
void Joueur::setNbProprietesMock (
    int n )
```

Utilisé par [CarteImpotMinijeu](#) dans les tests.

Définition à la ligne 101 du fichier Joueur.cpp.

Références nb_proprietes_mock, et use_mock_nb.

5.15.3.28 setPosition()

```
void Joueur::setPosition (
    int index )
```

Définition à la ligne 110 du fichier Joueur.cpp.

Références position.

Référencé par CarteDeplacement::action().

5.15.3.29 tour()

```
void Joueur::tour ( ) [virtual]
```

Définition à la ligne 112 du fichier Joueur.cpp.

Références compteur_tours.

5.15.4 Documentation des données membres

5.15.4.1 capital

```
int Joueur::capital [protected]
```

Définition à la ligne 70 du fichier Joueur.hpp.

Référencé par capitalSolvabilite(), conditionfinanciere(), donnerCapital(), enleverCapital(), getCapital(), misebanqueroute(), et setCapital().

5.15.4.2 cartes

```
std::vector<Carte*> Joueur::cartes [protected]
```

Définition à la ligne 78 du fichier Joueur.hpp.

Référencé par donnerCarte(), getNbCartes(), possedeCarte(), et retirerCarte().

5.15.4.3 case_actuelle

```
Case* Joueur::case_actuelle [protected]
```

Définition à la ligne 80 du fichier Joueur.hpp.

Référencé par bougerjoueur(), et getCaseActuelle().

5.15.4.4 compteur_double

```
int Joueur::compteur_double [protected]
```

Définition à la ligne 74 du fichier Joueur.hpp.

5.15.4.5 compteur_tours

```
int Joueur::compteur_tours [protected]
```

Définition à la ligne 75 du fichier Joueur.hpp.

Référencé par `getCompteurTours()`, `resetCompteurTours()`, et `tour()`.

5.15.4.6 dernier_lancer

```
int Joueur::dernier_lancer [protected]
```

Définition à la ligne 73 du fichier Joueur.hpp.

Référencé par `getDernierLancer()`, et `setDernierLancer()`.

5.15.4.7 nb_proprietes_mock

```
int Joueur::nb_proprietes_mock [protected]
```

Pour [CarteImpotMinijeu](#) tests.

Définition à la ligne 76 du fichier Joueur.hpp.

Référencé par `getNbProprietes()`, et `setNbProprietesMock()`.

5.15.4.8 nom

```
std::string Joueur::nom [protected]
```

Définition à la ligne 69 du fichier Joueur.hpp.

Référencé par `getNom()`.

5.15.4.9 position

```
int Joueur::position [protected]
```

Définition à la ligne 72 du fichier Joueur.hpp.

Référencé par `bougerjoueur()`, `getPosition()`, et `setPosition()`.

5.15.4.10 proprietes

```
std::vector<CasePropriete*> Joueur::proprietes [protected]
```

Définition à la ligne 79 du fichier Joueur.hpp.

Référencé par `acheter()`, `clearProprietes()`, `getNbProprietes()`, `getProprietes()`, et `hypotequer()`.

5.15.4.11 use_mock_nb

```
bool Joueur::use_mock_nb [protected]
```

Définition à la ligne 77 du fichier Joueur.hpp.

Référencé par `clearProprietes()`, `getNbProprietes()`, et `setNbProprietesMock()`.

5.15.4.12 val_propriete

```
int Joueur::val_propriete [protected]
```

Définition à la ligne 71 du fichier Joueur.hpp.

Référencé par `acheter()`, `capitalSolvabilite()`, `clearProprietes()`, `conditionfinanciere()`, `getValPropriete()`, et `hypotequer()`.

La documentation de cette classe a été générée à partir du fichier suivant :

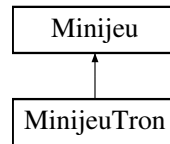
- [Joueur.hpp](#)
- [Joueur.cpp](#)

5.16 Référence de la classe Minijeu

Mini-jeu jouable pour 2, 3 ou 4 joueurs.

```
#include <Minijeu.hpp>
```

Graphe d'héritage de Minijeu:



Fonctions membres publiques

- `Minijeu()`
- `virtual ~Minijeu()`=default
- `virtual const char * getNom()` const
- `virtual Joueur * jouer(Joueur *j1, Joueur *j2)`
Duel 2 joueurs → retourne le gagnant.
- `virtual Joueur ** jouer(Joueur *j1, Joueur *j2, Joueur *j3)`
3 joueurs → tableau[3] trié du 1er au 3e, sans doublons.
- `virtual Joueur ** jouer(Joueur *j1, Joueur *j2, Joueur *j3, Joueur *j4)`
4 joueurs → tableau[4] trié du 1er au 4e, sans doublons.

5.16.1 Description détaillée

Mini-jeu jouable pour 2, 3 ou 4 joueurs.

L'implémentation de base utilise un tirage aléatoire. Les mini-jeux graphiques (Tron, Pong...) héritent de cette classe et surchargent les méthodes `jouer()`.

Définition à la ligne 19 du fichier Minijeu.hpp.

5.16.2 Documentation des constructeurs et destructeur

5.16.2.1 Minijeu()

```
Minijeu::Minijeu ( )
```

Définition à la ligne 11 du fichier Minijeu.cpp.

5.16.2.2 ~Minijeu()

```
virtual Minijeu::~~Minijeu ( ) [virtual], [default]
```

5.16.3 Documentation des fonctions membres

5.16.3.1 getNom()

```
const char * Minijeu::getNom ( ) const [virtual]
```

Réimplémentée dans `MinijeuTron`.

Définition à la ligne 12 du fichier Minijeu.cpp.

5.16.3.2 jouer() [1/3]

```
Joueur * MiniJeu::jouer (
    Joueur * j1,
    Joueur * j2 ) [virtual]
```

Duel 2 joueurs → retourne le gagnant.

Postcondition

résultat != nullptr; résultat == j1 || résultat == j2

Réimplémentée dans [MiniJeuTron](#).

Définition à la ligne 14 du fichier MiniJeu.cpp.

Référencé par CaseEvenement::action().

5.16.3.3 jouer() [2/3]

```
Joueur ** MiniJeu::jouer (
    Joueur * j1,
    Joueur * j2,
    Joueur * j3 ) [virtual]
```

3 joueurs → tableau[3] trié du 1er au 3e, sans doublons.

Avertissement

L'appelant libère le tableau avec delete[].

Définition à la ligne 16 du fichier MiniJeu.cpp.

5.16.3.4 jouer() [3/3]

```
Joueur ** MiniJeu::jouer (
    Joueur * j1,
    Joueur * j2,
    Joueur * j3,
    Joueur * j4 ) [virtual]
```

4 joueurs → tableau[4] trié du 1er au 4e, sans doublons.

Avertissement

L'appelant libère le tableau avec delete[].

Définition à la ligne 24 du fichier MiniJeu.cpp.

La documentation de cette classe a été générée à partir du fichier suivant :

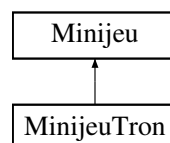
- [MiniJeu.hpp](#)
- [MiniJeu.cpp](#)

5.17 Référence de la classe MiniJeuTron

Hérite de [MiniJeu](#). Ouvre une fenêtre SFML pour un duel 1v1.

```
#include <MiniJeuTron.hpp>
```

Graphe d'héritage de MiniJeuTron:



Fonctions membres publiques

- [MinijeuTron](#) ()
- [~MinijeuTron](#) () override=default
- const char * [getNom](#) () const override
- [Joueur](#) * [jouer](#) ([Joueur](#) *j1, [Joueur](#) *j2) override
Lance la fenêtre SFML, gère la partie, et retourne le vainqueur.

5.17.1 Description détaillée

Hérite de [Minijeu](#). Ouvre une fenêtre SFML pour un duel 1v1.
Définition à la ligne 20 du fichier MinijeuTron.hpp.

5.17.2 Documentation des constructeurs et destructeur

5.17.2.1 MinijeuTron()

`MinijeuTron::MinijeuTron ( )`
Définition à la ligne 36 du fichier MinijeuTron.cpp.

5.17.2.2 ~MinijeuTron()

`MinijeuTron::~~MinijeuTron ( ) [override], [default]`

5.17.3 Documentation des fonctions membres

5.17.3.1 getNom()

`const char * MinijeuTron::getNom ( ) const [override], [virtual]`
Réimplémentée à partir de [Minijeu](#).
Définition à la ligne 38 du fichier MinijeuTron.cpp.

5.17.3.2 jouer()

`Joueur * MinijeuTron::jouer (`
 `Joueur * j1,`
 `Joueur * j2 ) [override], [virtual]`

Lance la fenêtre SFML, gère la partie, et retourne le vainqueur.

Paramètres

<i>j1</i>	Joueur 1 (contrôle Rouge - Flèches)
<i>j2</i>	Joueur 2 (contrôle Vert - ZQSD)

Réimplémentée à partir de [Minijeu](#).
Définition à la ligne 54 du fichier MinijeuTron.cpp.
Références `Joueur::getNom()`.
La documentation de cette classe a été générée à partir du fichier suivant :

- [MinijeuTron.hpp](#)
- [MinijeuTron.cpp](#)

5.18 Référence de la classe Plateau

Chef d'orchestre de la partie GambleStyle.

```
#include <Plateau.hpp>
```

Fonctions membres publiques

- [Plateau](#) ()
- [~Plateau](#) ()
- void [ajouterJoueur](#) ([Joueur](#) *j)
- void [retirerJoueursEnFaillite](#) ()
- void [donnercapital](#) ([Joueur](#) *j, int montant)
- void [enlevercapital](#) ([Joueur](#) *j, int montant)
- void [transfertargent](#) ([Joueur](#) *j1, [Joueur](#) *j2, int montant)
- void [ordre](#) ()
- void [trierParDernierLancer](#) ()
- void [update](#) ()
- void [gamelooping](#) ()
- bool [finDePartie](#) () const
- [Joueur](#) * [fin](#) () const
- [Joueur](#) * [getJoueur](#) (int index) const
- int [getNbJoueursActifs](#) () const
- [Case](#) * [getCase](#) (int index) const
- int [getNbCases](#) () const

5.18.1 Description détaillée

Chef d'orchestre de la partie GambleStyle.

Définition à la ligne 18 du fichier Plateau.hpp.

5.18.2 Documentation des constructeurs et destructeur

5.18.2.1 Plateau()

```
Plateau::Plateau ( )
```

Définition à la ligne 11 du fichier Plateau.cpp.

5.18.2.2 ~Plateau()

```
Plateau::~~Plateau ( )
```

Définition à la ligne 13 du fichier Plateau.cpp.

5.18.3 Documentation des fonctions membres

5.18.3.1 ajouterJoueur()

```
void Plateau::ajouterJoueur (
    Joueur * j )
```

Définition à la ligne 31 du fichier Plateau.cpp.

Référencé par main().

5.18.3.2 donnercapital()

```
void Plateau::donnercapital (
    Joueur * j,
    int montant )
```

Définition à la ligne 48 du fichier Plateau.cpp.

Références Joueur::donnerCapital().
Référéncé par CarteGain::action().

5.18.3.3 enlevercapital()

```
void Plateau::enlevercapital (
    Joueur * j,
    int montant )
```

Définition à la ligne 49 du fichier Plateau.cpp.

Références Joueur::enleverCapital().

Référéncé par CarteAmende::action(), et CarteImpotMinijeu::action().

5.18.3.4 fin()

```
Joueur * Plateau::fin ( ) const
```

Définition à la ligne 80 du fichier Plateau.cpp.

Références Joueur::capitalSolvabilite().

Référéncé par main().

5.18.3.5 finDePartie()

```
bool Plateau::finDePartie ( ) const
```

Définition à la ligne 75 du fichier Plateau.cpp.

Référéncé par gamelooping().

5.18.3.6 gamelooping()

```
void Plateau::gamelooping ( )
```

Définition à la ligne 73 du fichier Plateau.cpp.

Références finDePartie(), et update().

Référéncé par main().

5.18.3.7 getCase()

```
Case * Plateau::getCase (
    int index ) const
```

Définition à la ligne 97 du fichier Plateau.cpp.

Référéncé par CarteDeplacement::action().

5.18.3.8 getJoueur()

```
Joueur * Plateau::getJoueur (
    int index ) const
```

Définition à la ligne 93 du fichier Plateau.cpp.

Référéncé par CarteAnniversaire::action().

5.18.3.9 getNbCases()

```
int Plateau::getNbCases ( ) const
```

Définition à la ligne 100 du fichier Plateau.cpp.

5.18.3.10 getNbJoueursActifs()

```
int Plateau::getNbJoueursActifs ( ) const
```

Définition à la ligne 96 du fichier Plateau.cpp.

Référencé par CarteAnniversaire::action().

5.18.3.11 ordre()

```
void Plateau::ordre ( )
```

Définition à la ligne 54 du fichier Plateau.cpp.

Références trierParDernierLancer().

Référencé par main().

5.18.3.12 retirerJoueursEnFaillite()

```
void Plateau::retirerJoueursEnFaillite ( )
```

Définition à la ligne 40 du fichier Plateau.cpp.

5.18.3.13 transfertargent()

```
void Plateau::transfertargent (
```

```
    Joueur * j1,
```

```
    Joueur * j2,
```

```
    int montant )
```

Définition à la ligne 50 du fichier Plateau.cpp.

Références Joueur::donnerCapital(), et Joueur::enleverCapital().

Référencé par CarteAnniversaire::action().

5.18.3.14 trierParDernierLancer()

```
void Plateau::trierParDernierLancer ( )
```

Définition à la ligne 59 du fichier Plateau.cpp.

Référencé par ordre().

5.18.3.15 update()

```
void Plateau::update ( )
```

Définition à la ligne 65 du fichier Plateau.cpp.

Références FAILLITE.

Référencé par gamelooping().

La documentation de cette classe a été générée à partir du fichier suivant :

- [Plateau.hpp](#)
- [Plateau.cpp](#)

Chapitre 6

Documentation des fichiers

6.1 Référence du fichier Carte.cpp

Implémentation de la classe de base [Carte](#).

```
#include "Carte.hpp"
```

6.1.1 Description détaillée

Implémentation de la classe de base [Carte](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.2 Référence du fichier Carte.hpp

Classe abstraite [Carte](#).

```
#include <string>
```

Classes

— class [Carte](#)

[Carte](#) tirée sur une [CaseAleatoire](#). Chaque sous-classe implémente [action\(\)](#).

6.2.1 Description détaillée

Classe abstraite [Carte](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.3 Référence du fichier CarteAmende.cpp

Implémentation de [CarteAmende](#).

```
#include "CarteAmende.hpp"
```

```
#include "Joueur.hpp"
```

```
#include "Plateau.hpp"
```

6.3.1 Description détaillée

Implémentation de [CarteAmende](#).

6.4 Référence du fichier CarteAmende.hpp

Classe [CarteAmende](#).

```
#include "Carte.hpp"
```

Classes

- class [CarteAmende](#)
Retire un montant fixe du capital du joueur.

6.4.1 Description détaillée

Classe [CarteAmende](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.5 Référence du fichier CarteAnniversaire.cpp

Implémentation de [CarteAnniversaire](#).

```
#include "CarteAnniversaire.hpp"  
#include "Joueur.hpp"  
#include "Plateau.hpp"
```

6.5.1 Description détaillée

Implémentation de [CarteAnniversaire](#).

6.6 Référence du fichier CarteAnniversaire.hpp

Classe [CarteAnniversaire](#).

```
#include "Carte.hpp"
```

Classes

- class [CarteAnniversaire](#)
Chaque autre joueur verse un montant fixe au titulaire.

6.6.1 Description détaillée

Classe [CarteAnniversaire](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.7 Référence du fichier CarteDeplacement.cpp

Implémentation de [CarteDeplacement](#).

```
#include "CarteDeplacement.hpp"  
#include "Joueur.hpp"  
#include "Plateau.hpp"  
#include "Case.hpp"
```

6.7.1 Description détaillée

Implémentation de [CarteDeplacement](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.8 Référence du fichier CarteDeplacement.hpp

Classe [CarteDeplacement](#).

```
#include "Carte.hpp"
```

Classes

- class [CarteDeplacement](#)
Déplace un joueur vers une case spécifique.

6.8.1 Description détaillée

Classe [CarteDeplacement](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.9 Référence du fichier CarteDoubleGain.cpp

Implémentation de [CarteDoubleGain](#).

```
#include "CarteDoubleGain.hpp"  
#include "Joueur.hpp"
```

6.9.1 Description détaillée

Implémentation de [CarteDoubleGain](#).

6.10 Référence du fichier CarteDoubleGain.hpp

Classe [CarteDoubleGain](#).

```
#include "Carte.hpp"
```

Classes

- class [CarteDoubleGain](#)
Double les gains du joueur lors de sa prochaine victoire à un mini-jeu.

6.10.1 Description détaillée

Classe [CarteDoubleGain](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.11 Référence du fichier CarteGain.cpp

Implémentation de [CarteGain](#).

```
#include "CarteGain.hpp"
#include "Joueur.hpp"
#include "Plateau.hpp"
```

6.11.1 Description détaillée

Implémentation de [CarteGain](#).

6.12 Référence du fichier CarteGain.hpp

Classe [CarteGain](#).

```
#include "Carte.hpp"
```

Classes

- class [CarteGain](#)
Crédite un montant fixe au capital du joueur.

6.12.1 Description détaillée

Classe [CarteGain](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.13 Référence du fichier CartelImpotMinijeux.cpp

Implémentation de [CartelImpotMinijeux](#).

```
#include "CarteImpotMinijeux.hpp"
#include "Joueur.hpp"
#include "Plateau.hpp"
```

6.13.1 Description détaillée

Implémentation de [CartelImpotMinijeux](#).

6.14 Référence du fichier CartelImpotMinijeux.hpp

Classe [CartelImpotMinijeux](#).

```
#include "Carte.hpp"
```

Classes

- class [CartelImpotMinijeux](#)
Prélève $\text{tauxParCase} \times \text{nb_cases_possédées}$.

6.14.1 Description détaillée

Classe [CarteImpotMinijeu](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.15 Référence du fichier CarteSortiePrison.cpp

Implémentation de [CarteSortiePrison](#).

```
#include "CarteSortiePrison.hpp"
```

```
#include "Joueur.hpp"
```

6.15.1 Description détaillée

Implémentation de [CarteSortiePrison](#).

6.16 Référence du fichier CarteSortiePrison.hpp

Classe [CarteSortiePrison](#).

```
#include "Carte.hpp"
```

Classes

- class [CarteSortiePrison](#)
Permet au joueur de sortir de prison sans payer ni lancer les dés.

6.16.1 Description détaillée

Classe [CarteSortiePrison](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.17 Référence du fichier Case.cpp

Implémentation de la classe de base [Case](#).

```
#include "Case.hpp"
```

6.17.1 Description détaillée

Implémentation de la classe de base [Case](#).

6.18 Référence du fichier Case.hpp

Classe abstraite [Case](#).

```
#include <string>
```

Classes

- class [Case](#)
[Case](#) abstraite du plateau. Chaque sous-classe implémente [action\(\)](#).

6.18.1 Description détaillée

Classe abstraite [Case](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.19 Référence du fichier CaseAleatoire.cpp

Implémentation de [CaseAleatoire](#).

```
#include "CaseAleatoire.hpp"
#include "Carte.hpp"
```

6.19.1 Description détaillée

Implémentation de [CaseAleatoire](#).

6.20 Référence du fichier CaseAleatoire.hpp

Classe [CaseAleatoire](#).

```
#include "Case.hpp"
#include <vector>
```

Classes

— class [CaseAleatoire](#)
Tire la première carte d'une file circulaire et l'applique.

6.20.1 Description détaillée

Classe [CaseAleatoire](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.21 Référence du fichier CaseDepart.cpp

Implémentation de [CaseDepart](#).

```
#include "CaseDepart.hpp"
#include "Joueur.hpp"
```

6.21.1 Description détaillée

Implémentation de [CaseDepart](#).

6.22 Référence du fichier CaseDepart.hpp

Classe [CaseDepart](#).

```
#include "Case.hpp"
```


Classes

- class [CaseDepart](#)
Crédite un montant fixe à chaque passage.

6.22.1 Description détaillée

Classe [CaseDepart](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.23 Référence du fichier CaseEvenement.cpp

Implémentation de [CaseEvenement](#).

```
#include "CaseEvenement.hpp"  
#include "Joueur.hpp"  
#include "MinijeuTron.hpp"  
#include <cstdlib>
```

6.23.1 Description détaillée

Implémentation de [CaseEvenement](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.24 Référence du fichier CaseEvenement.hpp

Classe [CaseEvenement](#).

```
#include "Case.hpp"  
#include <vector>
```

Classes

- class [CaseEvenement](#)
Déclenche un mini-jeu pour tous les joueurs. Distribution 3/4 – 1/4.

6.24.1 Description détaillée

Classe [CaseEvenement](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.25 Référence du fichier CasePrison.cpp

Implémentation de [CasePrison](#).

```
#include "CasePrison.hpp"  
#include "Joueur.hpp"  
#include "CarteSortiePrison.hpp"  
#include <cstdlib>
```

6.25.1 Description détaillée

Implémentation de [CasePrison](#).

6.26 Référence du fichier CasePrison.hpp

Classe [CasePrison](#).

```
#include "Case.hpp"
#include <vector>
#include <utility>
```

Classes

- class [CasePrison](#)
Bloque le joueur jusqu'à double, carte, ou 3 tours.

6.26.1 Description détaillée

Classe [CasePrison](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.27 Référence du fichier CasePropriete.cpp

Implémentation de [CasePropriete](#).

```
#include "CasePropriete.hpp"
#include "Joueur.hpp"
```

6.27.1 Description détaillée

Implémentation de [CasePropriete](#).

6.28 Référence du fichier CasePropriete.hpp

Classe [CasePropriete](#).

```
#include "Case.hpp"
#include <string>
```

Classes

- class [CasePropriete](#)
[Case](#) achetable avec système de duel (gamble).

6.28.1 Description détaillée

Classe [CasePropriete](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.29 Référence du fichier Condition.hpp

Énumération de l'état financier d'un [Joueur](#).

Énumérations

- enum class `Condition` { `RICHE` , `BANQUEROUTE` , `FAILLITE` }
État financier d'un joueur.

6.29.1 Description détaillée

Énumération de l'état financier d'un `Joueur`.

Projet GambleStyle — GM4 INSA Rouen Normandie

6.29.2 Documentation du type de l'énumération

6.29.2.1 Condition

enum `Condition` [strong]
État financier d'un joueur.

Valeurs énumérées

<code>RICHE</code>	capital $\geq$ 0
<code>BANQUEROUTE</code>	capital < 0, val_proprietes > 0
<code>FAILLITE</code>	capital de solvabilite $\leq$ 0, joueur éliminé

Définition à la ligne 13 du fichier Condition.hpp.

6.30 Référence du fichier Joueur.cpp

Implémentation de la classe `Joueur`.

```
#include "Joueur.hpp"
#include "Case.hpp"
#include "Carte.hpp"
#include "CasePropriete.hpp"
#include <cstdlib>
#include <algorithm>
```

6.30.1 Description détaillée

Implémentation de la classe `Joueur`.

6.31 Référence du fichier Joueur.hpp

Déclaration de la classe `Joueur`.

```
#include <string>
#include <vector>
#include "Condition.hpp"
```

Classes

- class `Joueur`
Représente un participant au jeu GambleStyle.

6.31.1 Description détaillée

Déclaration de la classe [Joueur](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.32 Référence du fichier main.cpp

Point d'entrée du jeu GambleStyle.

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "Plateau.hpp"
#include "Joueur.hpp"
```

Fonctions

— int [main](#) ()

6.32.1 Description détaillée

Point d'entrée du jeu GambleStyle.

6.32.2 Documentation des fonctions

6.32.2.1 main()

```
int main ( )
```

Définition à la ligne 11 du fichier main.cpp.

Références [Plateau::ajouterJoueur\(\)](#), [Plateau::fin\(\)](#), [Plateau::gamelooping\(\)](#), [Joueur::getNom\(\)](#), et [Plateau::ordre\(\)](#).

6.33 Référence du fichier Minijeu.cpp

Implémentation de [Minijeu](#) (version aléatoire).

```
#include "Minijeu.hpp"
#include "Joueur.hpp"
#include <cstdlib>
#include <algorithm>
#include <vector>
```

6.33.1 Description détaillée

Implémentation de [Minijeu](#) (version aléatoire).

6.34 Référence du fichier Minijeu.hpp

Classe [Minijeu](#) (implémentation par défaut aléatoire).

Classes

— class [Minijeu](#)
Mini-jeu jouable pour 2, 3 ou 4 joueurs.

6.34.1 Description détaillée

Classe [Minijeu](#) (implémentation par défaut aléatoire).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.35 Référence du fichier MinijeuTron.cpp

Implémentation du mini-jeu Tron en SFML.

```
#include "MinijeuTron.hpp"
#include "Joueur.hpp"
#include <iostream>
```

6.35.1 Description détaillée

Implémentation du mini-jeu Tron en SFML.

Projet GambleStyle — GM4 INSA Rouen Normandie

6.36 Référence du fichier MinijeuTron.hpp

Classe [MinijeuTron](#) implémentant le jeu Tron via SFML.

```
#include "Minijeu.hpp"
#include <SFML/Graphics.hpp>
#include <vector>
#include <cstdint>
```

Classes

— class [MinijeuTron](#)
Hérite de [Minijeu](#). Ouvre une fenêtre SFML pour un duel 1v1.

6.36.1 Description détaillée

Classe [MinijeuTron](#) implémentant le jeu Tron via SFML.

Projet GambleStyle — GM4 INSA Rouen Normandie

6.37 Référence du fichier Plateau.cpp

Implémentation de la classe [Plateau](#).

```
#include "Plateau.hpp"
#include "Joueur.hpp"
#include "CaseDepart.hpp"
#include "CasePropriete.hpp"
#include <algorithm>
```

6.37.1 Description détaillée

Implémentation de la classe [Plateau](#).

6.38 Référence du fichier Plateau.hpp

Déclaration de la classe [Plateau](#).

```
#include <vector>
```

Classes

- class [Plateau](#)
Chef d'orchestre de la partie GambleStyle.

6.38.1 Description détaillée

Déclaration de la classe [Plateau](#).

Projet GambleStyle — GM4 INSA Rouen Normandie

6.39 Référence du fichier README.md

Index

- ~Carte
 - Carte, [9](#)
- ~Case
 - Case, [18](#)
- ~CaseAleatoire
 - CaseAleatoire, [20](#)
- ~CaseEvenement
 - CaseEvenement, [22](#)
- ~Joueur
 - Joueur, [29](#)
- ~Minijeu
 - Minijeu, [35](#)
- ~MinijeuTron
 - MinijeuTron, [37](#)
- ~Plateau
 - Plateau, [38](#)
- achatPropose
 - CasePropriete, [26](#)
- acheter
 - Joueur, [29](#)
- action
 - Carte, [10](#)
 - CarteAmende, [11](#)
 - CarteAnniversaire, [12](#)
 - CarteDeplacement, [13](#)
 - CarteDoubleGain, [14](#)
 - CarteGain, [15](#)
 - CarteImpotMinijeu, [15](#)
 - CarteSortiePrison, [16](#)
 - Case, [18](#)
 - CaseAleatoire, [20](#)
 - CaseDepart, [21](#)
 - CaseEvenement, [22](#)
 - CasePrison, [24](#)
 - CasePropriete, [26](#)
- ajouterCarte
 - CaseAleatoire, [20](#)
- ajouterJoueur
 - CaseEvenement, [22](#)
 - Plateau, [38](#)
- BANQUEROUTE
 - Condition.hpp, [49](#)
- bougerjoueur
 - Joueur, [29](#)
- capital
 - Joueur, [33](#)
- capitalSolvabilite
 - Joueur, [29](#)
- Carte, [9](#)
 - ~Carte, [9](#)
 - action, [10](#)
 - Carte, [9](#)
 - getId, [10](#)
 - getNom, [10](#)
 - getTexte, [10](#)
 - nom, [10](#)
 - texte, [10](#)
- Carte.cpp, [41](#)
- Carte.hpp, [41](#)
- CarteAmende, [10](#)
 - action, [11](#)
 - CarteAmende, [11](#)
- CarteAmende.cpp, [41](#)
- CarteAmende.hpp, [42](#)
- CarteAnniversaire, [11](#)
 - action, [12](#)
 - CarteAnniversaire, [12](#)
- CarteAnniversaire.cpp, [42](#)
- CarteAnniversaire.hpp, [42](#)
- CarteDeplacement, [12](#)
 - action, [13](#)
 - CarteDeplacement, [13](#)
- CarteDeplacement.cpp, [42](#)
- CarteDeplacement.hpp, [43](#)
- CarteDoubleGain, [13](#)
 - action, [14](#)
 - CarteDoubleGain, [14](#)
- CarteDoubleGain.cpp, [43](#)
- CarteDoubleGain.hpp, [43](#)
- CarteGain, [14](#)
 - action, [15](#)
 - CarteGain, [14](#)
- CarteGain.cpp, [44](#)
- CarteGain.hpp, [44](#)
- CarteImpotMinijeu, [15](#)
 - action, [15](#)
 - CarteImpotMinijeu, [15](#)
- CarteImpotMinijeu.cpp, [44](#)
- CarteImpotMinijeu.hpp, [44](#)
- cartes
 - Joueur, [33](#)
- CarteSortiePrison, [16](#)
 - action, [16](#)
 - CarteSortiePrison, [16](#)
 - getId, [16](#)
 - ID, [17](#)

CarteSortiePrison.cpp, 45
 CarteSortiePrison.hpp, 45
 Case, 17
 ~Case, 18
 action, 18
 Case, 17, 18
 getIndex, 18
 getJoueurActif, 18
 getNom, 18
 index, 19
 joueur_actif, 19
 nom, 19
 setIndex, 18
 setJoueurActif, 18
 Case.cpp, 45
 Case.hpp, 45
 case_actuelle
 Joueur, 33
 CaseAleatoire, 19
 ~CaseAleatoire, 20
 action, 20
 ajouterCarte, 20
 CaseAleatoire, 20
 getPremiereCarteDeFile, 20
 CaseAleatoire.cpp, 46
 CaseAleatoire.hpp, 46
 CaseDepart, 20
 action, 21
 CaseDepart, 21
 getMontant, 21
 CaseDepart.cpp, 46
 CaseDepart.hpp, 46
 CaseEvenement, 21
 ~CaseEvenement, 22
 action, 22
 ajouterJoueur, 22
 CaseEvenement, 22
 choixminijeu, 23
 clearMinijeu, 23
 distribution, 23
 minijeuJoue, 23
 setClassement, 23
 setGagnant, 23
 setMise, 23
 CaseEvenement.cpp, 47
 CaseEvenement.hpp, 47
 CasePrison, 23
 action, 24
 CasePrison, 24
 emprisonner, 24
 forcerResultatDes, 24
 getTourPrison, 25
 setUtiliserCarteReponse, 25
 CasePrison.cpp, 47
 CasePrison.hpp, 48
 CasePropriete, 25
 achatPropose, 26
 action, 26
 CasePropriete, 26
 gamble, 26
 getGamble, 26
 getPrix, 26
 getPrixHypothèque, 26
 getProprietaire, 27
 isAchetable, 27
 minijeuLance, 27
 repartition, 27
 setAchetable, 27
 setGagnant, 27
 setGamble, 27
 setGamblePredefini, 27
 setMiseLimites, 28
 setProprietaire, 28
 CasePropriete.cpp, 48
 CasePropriete.hpp, 48
 choixminijeu
 CaseEvenement, 23
 clearMinijeu
 CaseEvenement, 23
 clearProprietes
 Joueur, 29
 compteur_double
 Joueur, 33
 compteur_tours
 Joueur, 33
 Condition
 Condition.hpp, 49
 Condition.hpp, 48
 BANQUEROUTE, 49
 Condition, 49
 FAILLITE, 49
 RICHE, 49
 conditionfinanciere
 Joueur, 30

 dernier_lancer
 Joueur, 34
 distribution
 CaseEvenement, 23
 donnerCapital
 Joueur, 30
 donnercapital
 Plateau, 38
 donnerCarte
 Joueur, 30

 emprisonner
 CasePrison, 24
 enleverCapital
 Joueur, 30
 enlevercapital
 Plateau, 39

 FAILLITE
 Condition.hpp, 49
 fin
 Plateau, 39

finDePartie
 Plateau, 39
forcerResultatDes
 CasePrison, 24

gamble
 CasePropriete, 26
gamelooing
 Plateau, 39
getCapital
 Joueur, 30
getCase
 Plateau, 39
getCaseActuelle
 Joueur, 30
getCompteurTours
 Joueur, 30
getDernierLancer
 Joueur, 31
getGamble
 CasePropriete, 26
getId
 Carte, 10
 CarteSortiePrison, 16
getIndex
 Case, 18
getJoueur
 Plateau, 39
getJoueurActif
 Case, 18
getMontant
 CaseDepart, 21
getNbCartes
 Joueur, 31
getNbCases
 Plateau, 39
getNbJoueursActifs
 Plateau, 39
getNbProprietes
 Joueur, 31
getNom
 Carte, 10
 Case, 18
 Joueur, 31
 Minijeu, 35
 MinijeuTron, 37
getPosition
 Joueur, 31
getPremiereCarteDeFile
 CaseAleatoire, 20
getPrix
 CasePropriete, 26
getPrixHypothèque
 CasePropriete, 26
getProprietaire
 CasePropriete, 27
getProprietes
 Joueur, 31
getTexte
 Carte, 10
getTourPrison
 CasePrison, 25
getValPropriete
 Joueur, 31

hypotequer
 Joueur, 31

ID
 CarteSortiePrison, 17
index
 Case, 19
isAchetable
 CasePropriete, 27

jouer
 Minijeu, 35, 36
 MinijeuTron, 37
Joueur, 28
 ~Joueur, 29
 acheter, 29
 bougerjoueur, 29
 capital, 33
 capitalSolvabilite, 29
 cartes, 33
 case_actuelle, 33
 clearProprietes, 29
 compteur_double, 33
 compteur_tours, 33
 conditionfinanciere, 30
 dernier_lancer, 34
 donnerCapital, 30
 donnerCarte, 30
 enleverCapital, 30
 getCapital, 30
 getCaseActuelle, 30
 getCompteurTours, 30
 getDernierLancer, 31
 getNbCartes, 31
 getNbProprietes, 31
 getNom, 31
 getPosition, 31
 getProprietes, 31
 getValPropriete, 31
 hypotequer, 31
 Joueur, 29
 lancerde, 32
 misebanqueroute, 32
 nb_proprietes_mock, 34
 nom, 34
 position, 34
 possedeCarte, 32
 proprietes, 34
 resetCompteurTours, 32
 retirerCarte, 32
 setCapital, 32
 setDernierLancer, 32
 setNbProprietesMock, 33

- setPosition, 33
 - tour, 33
 - use_mock_nb, 34
 - val_propriete, 34
- Joueur.cpp, 49
- Joueur.hpp, 49
- joueur_actif
 - Case, 19
- lancerde
 - Joueur, 32
- main
 - main.cpp, 50
- main.cpp, 50
 - main, 50
- Minijeu, 35
 - ~Minijeu, 35
 - getNom, 35
 - jouer, 35, 36
 - Minijeu, 35
- Minijeu.cpp, 50
- Minijeu.hpp, 50
- minijeuJoue
 - CaseEvenement, 23
- minijeuLance
 - CasePropriete, 27
- MinijeuTron, 36
 - ~MinijeuTron, 37
 - getNom, 37
 - jouer, 37
 - MinijeuTron, 37
- MinijeuTron.cpp, 51
- MinijeuTron.hpp, 51
- misebanqueroute
 - Joueur, 32
- nb_proprietes_mock
 - Joueur, 34
- nom
 - Carte, 10
 - Case, 19
 - Joueur, 34
- ordre
 - Plateau, 40
- Plateau, 38
 - ~Plateau, 38
 - ajouterJoueur, 38
 - donnercapital, 38
 - enlevercapital, 39
 - fin, 39
 - finDePartie, 39
 - gamelooping, 39
 - getCase, 39
 - getJoueur, 39
 - getNbCases, 39
 - getNbJoueursActifs, 39
 - ordre, 40
 - Plateau, 38
 - retirerJoueursEnFaillite, 40
 - transfertargent, 40
 - trierParDernierLancer, 40
 - update, 40
- Plateau.cpp, 51
- Plateau.hpp, 52
- position
 - Joueur, 34
- possedeCarte
 - Joueur, 32
- proprietes
 - Joueur, 34
- README.md, 52
- repartition
 - CasePropriete, 27
- resetCompteurTours
 - Joueur, 32
- retirerCarte
 - Joueur, 32
- retirerJoueursEnFaillite
 - Plateau, 40
- RICHE
 - Condition.hpp, 49
- setAchetable
 - CasePropriete, 27
- setCapital
 - Joueur, 32
- setClassement
 - CaseEvenement, 23
- setDernierLancer
 - Joueur, 32
- setGagnant
 - CaseEvenement, 23
 - CasePropriete, 27
- setGamble
 - CasePropriete, 27
- setGamblePredefini
 - CasePropriete, 27
- setIndex
 - Case, 18
- setJoueurActif
 - Case, 18
- setMise
 - CaseEvenement, 23
- setMiseLimites
 - CasePropriete, 28
- setNbProprietesMock
 - Joueur, 33
- setPosition
 - Joueur, 33
- setProprietaire
 - CasePropriete, 28
- setUtiliserCarteReponse
 - CasePrison, 25

texte
 Carte, [10](#)
tour
 Joueur, [33](#)
transfertargent
 Plateau, [40](#)
trierParDernierLancer
 Plateau, [40](#)

update
 Plateau, [40](#)
use_mock_nb
 Joueur, [34](#)

val_propriete
 Joueur, [34](#)